第三章 Skill架构设计、AI编排与评估框架

> **本章导言**
>
> 在第一章中，我们建立了技能（Skill）与氛围编程（Vibe Coding）的概念基础；在第二章中，我们学会了用五要素框架拆解设计研究方法，并将其映射到AI能力模块。然而，从“拆解结果”到“可运行的AI技能”之间，仍然横亘着一道鸿沟：如何将一堆松散的AI能力组织成一个结构清晰、人机协同、可评估、可交付的Skill？
>
> 本章正是为跨越这道鸿沟而设。我们将首先探讨Skill的三种核心架构模式——管道式、交互式与自主代理式，并深入分析“人类在环”的设计原则。其次，我们将学习AI编排的基础技术——链式提示、工具调用、检索增强生成与状态管理，掌握用自然语言描述复杂处理链的方法。再次，我们引入一个五维评估框架（准确性、一致性、可解释性、效率、体验），将设计研究中的信效度概念迁移到AI技能的评价中。最后，通过小组设计文档的撰写与Gradio原型的演示，完成从“概念设计”到“可验证原型”的闭环。
>
> 学完本章，你将不再只是“让AI做点分析”——你将能够**设计**、**编排**、**评估**一个完整的人机协同设计研究技能，为后续的开发冲刺奠定坚实的蓝图与方法论基础。

本章学习目标

完成本章学习后，读者应能够：

1. 区分管道式、交互式和自主代理式三种Skill架构模式，并能根据方法的特性做出合理选择；
2. 设计“人类在环”的交互节点，并选择合适的交互模式（审核、引导、协作、申诉）；
3. 使用自然语言描述多步骤AI处理链，应用链式提示、工具调用等编排技术；
4. 运用五维评估框架（准确性、一致性、可解释性、效率、体验）为Skill设计评估方案；
5. 撰写一份结构完整的Skill设计文档，并搭建Gradio原型界面用于交互验证。

3.1 三种核心Skill架构模式

3.1.1 从拆解到架构：为什么要选择架构？

在第二章中，你已经完成了对一个设计研究方法的五要素拆解，并绘制了步骤到AI能力的映射表。这些成果回答了一个问题：“这个方法由哪些部分组成？每个部分可以用什么AI技术？”然而，这些部分如何连接？处理过程中AI和人的角色如何分配？数据如何在步骤之间流动？这些问题的答案，就是Skill的**架构**。

我们可以用建筑类型来类比。一座房子可以有多种户型：大开间（所有功能区混在一起）、走廊串联式（客厅-走廊-卧室-书房线性排列）、或开放式布局加灵活隔断。同样的建筑材料（砖、木、玻璃），通过不同的架构组织，可以产生截然不同的居住体验。

在AI Skill设计中，架构决定了三个关键特性：① **处理流**（数据如何从输入流向输出）；② **控制权**（AI和人类分别在哪些节点拥有决策权）；③ **容错性**（当一个环节出错时，错误如何被检测和纠正）。因此，选择恰当的架构并非锦上添花，而是Skill能否真正服务于设计研究实践的根本保障。

本节将介绍三种核心架构模式——**管道式**、**交互式**与**自主代理式**。它们并非互斥，一个复杂的Skill可能在不同环节混合使用多种模式。但理解每种模式的本质，将帮助你做出理性的设计决策。

3.1.2 管道式架构

定义与结构

管道式架构是最简单、最直观的架构模式。它模拟了工业流水线：原料从一端进入，经过一系列固定的处理站，成品从另一端输出。中间没有分支、没有等待、没有人的干预。每个处理站只负责一道工序，前一道的输出是后一道的输入。

在Skill语境下，管道式架构的结构可以形式化地表示为：

```
输入 → 步骤₁(AI) → 步骤₂(AI) → 步骤₃(AI) → ... → 输出
```

其中每个步骤是一个可独立描述的AI能力（如文本分类、情感分析、聚类），步骤之间的连接是固定的、线性的。一旦启动，处理链自动运行至结束。

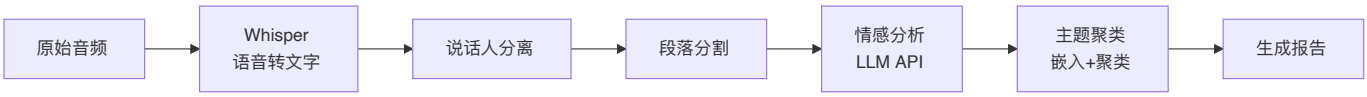


图3-1 访谈分析管道式架构示例

适用情境

管道式架构最适合以下两类场景：

第一类：高度标准化的分析任务。如果方法中的某个流程，每次处理的步骤都相同、每个步骤的输入输出格式明确、人在该流程中通常不做创造性判断而只是"照章办事"，那么它天然适合管道化。典型例子：问卷数据处理——导入CSV → 数据清洗 → 描述统计 → 开放题编码 → 交叉分析 → 报告输出。每一步都有统计学或NLP的标准化操作，无需研究者的中途判断。

第二类：AI准确率极高且出错代价低的任务。当AI在某类任务上的表现已经接近或超过人类水平（如清晰录音的语音转文字、标准新闻的情感分析），并且出错的后果可以轻易被事后检查发现和纠正时，管道化可以极大提升效率。例如，将10场访谈录音批量转录为文字，即使有个别错字，研究者也可以在阅读时轻易发现并修正。

优缺点分析

优点	缺点
结构简单，易于理解和实现	缺乏灵活性，无法处理未预见的边界情况
每个步骤可独立优化和替换	错误会沿管道传播，难以中途纠正
处理速度快，无等待延迟	AI难以进行全局一致性检查
适合大规模批量处理	若用户需要中间判断，管道式不适用

设计研究中的典型案例

- **访谈转录与分析管道**：录音 → 语音转文字 → 说话人分离 → 段落情感标注 → 关键词提取 → 摘要生成。这是一个完全自动化的管道，适用于研究者只需要快速浏览访谈要点而不需要深度诠释的场景。
- **问卷开放题聚类管道**：CSV导入 → 文本清洗 → 嵌入向量计算 → HDBSCAN聚类 → 生成每组代表性回复。研究者直接获得主题分组，无需参与聚类过程。

实现要点（Vibe Coding提示词模式）

在Vibe Coding中描述管道式架构时，关键是明确每个步骤的输入输出类型和连接逻辑。一个有效的提示词模板如下：

> 请写一个处理管道，包含以下步骤：
>
> - 步骤1：接收MP3音频文件，使用Whisper转录为文本，输出字符串
> - 步骤2：接收步骤1的文本，按句号分割为句子列表，输出列表
> - 步骤3：对每个句子调用DeepSeek API进行情感分析，输出包含`{sentence, sentiment, confidence}`的字典列表
> - 步骤4：接收情感分析结果，统计正面/负面/中性的比例，生成饼图数据
>
> 请将每个步骤实现为独立的Python函数，并用一个主函数按顺序串联它们。中间结果用变量传递，不要写入全局状态。

3.1.3 交互式架构

定义与结构

交互式架构在管道的基础上增加了“人类在环”（Human-in-the-Loop）的节点。它不再是单向的流水线，而更像是“AI起草，人编辑”的协作流程。AI先产出一份初稿，人类在关键节点审查、修改、确认；修改后的结果可能作为输入反馈给AI，触发新一轮处理。

交互式架构的形式化表示为：

```
输入 → 步骤₁(AI) → 人工节点 → 步骤₂(AI，基于修改) → 人工节点 → … → 输出
```

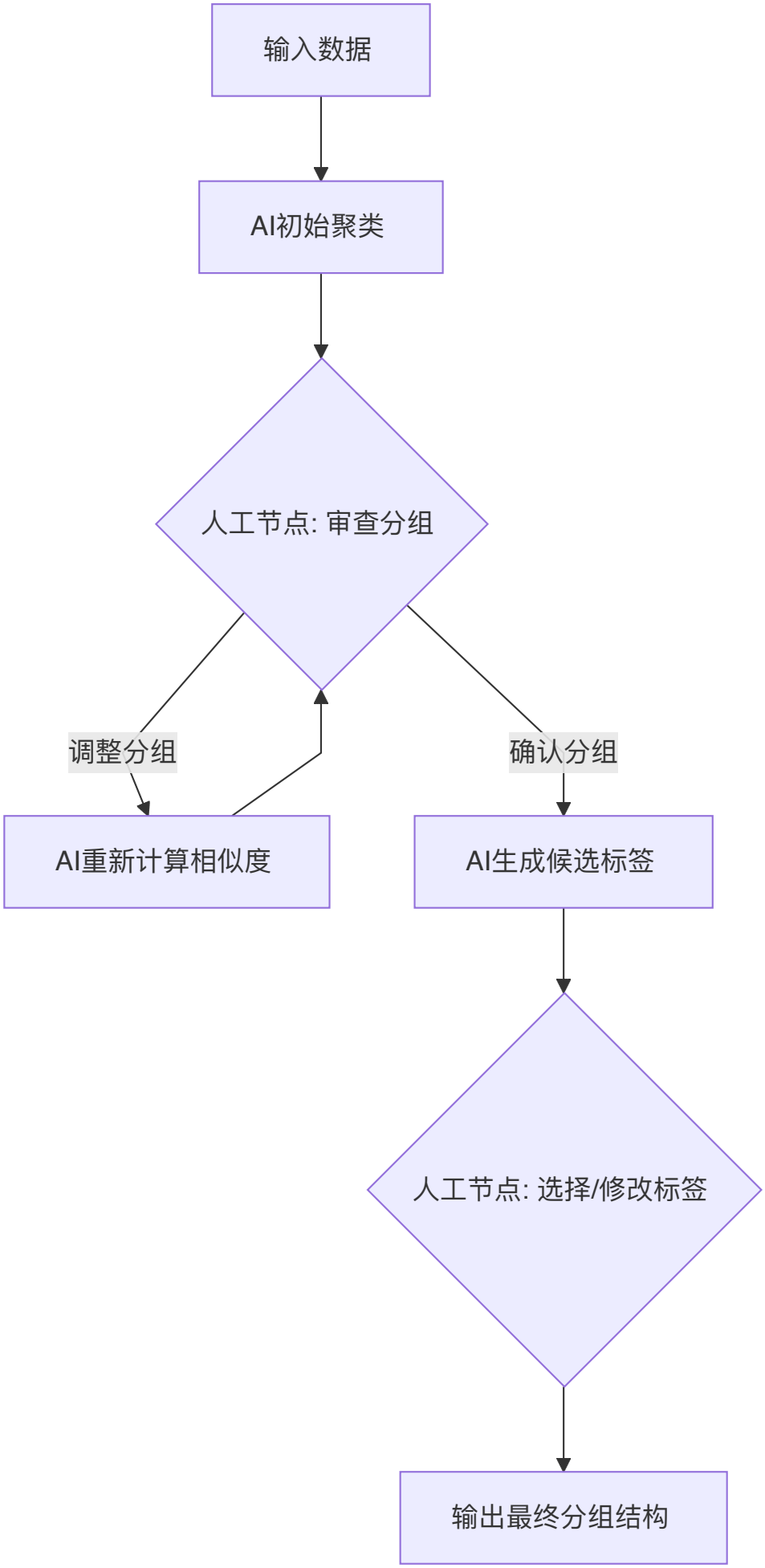

输入数据
AI初始聚类
人工节点: 审查分组
调整分组
确认分组
AI重新计算相似度
AI生成候选标签
人工节点: 选择/修改标签
输出最终分组结构

图3-2 亲和图交互式Skill架构示意图

人工节点可以有多种形态：**确认**（通过/驳回）、**修改**（编辑文本、调整参数）、**选择**（从多个AI候选中挑选一个）、**标注**（标记错误、补充信息）、**拖拽**（重新组织结构）。交互式架构的核心特征是：人的输入会改变AI后续处理的行为——仅仅"点击确认而不修改"也算改变，因为它告诉AI"这个结果是可接受的"。

适用情境

交互式架构是设计研究AI Skill中最推荐的架构模式。原因在于，设计研究的核心活动——诠释、命名、判断优先级——本质上是依赖研究者的隐性知识和项目语境，难以被完全算法化。AI可以做得很好（甚至比初学者更好），但把最终判断权完全交给AI是有风险的。交互式架构在效率与可靠性之间取得了最佳平衡。

具体而言，交互式架构最适合三类场景：

1. **AI产出质量不错但不完美**：编码建议、主题聚类、报告草稿——AI能做出70-80%质量的产出，但剩余的20-30%需要人来打磨。
2. **人的判断会显著影响下游结果**：在亲和图中，你决定将两个组合并还是分开，会改变整个分组结构；在创意筛选中，你标记哪些概念值得深入，决定了生成方向。
3. **输出需要承担专业责任**：当分析结果将影响产品决策时，研究者需要在最终输出上署名，因此必须有机会审查和修改。

优缺点分析

优点	缺点
保留人类判断在关键节点	开发复杂度高，需设计交互界面
提高输出质量和可信度	处理时间变长（等待人操作）
灵活应对AI的局限性	依赖用户具备方法知识
人机互补，1+1>2	交互设计不当可能导致"人给AI打工"

设计研究中的典型案例

- **智能亲和图Skill**：AI基于语义相似度建议初始分组 → 研究者拖拽调整条目的归属 → AI根据人的调整重新计算剩余条目的分组建议 → 研究者确认并修改分组命名 → AI为修改后的各组搜索补充引语。这里有两轮人工介入：分组调整、命名确认。
- **访谈分析辅助Skill**：AI对访谈稿进行段落编码 → 研究者审核编码，删除不恰当的、补充遗漏的 → AI基于修改后的编码重新提取主题 → 研究者确认或修改主题名称 → AI生成报告草稿，研究者最终润色。

"人类在环"的设计原则

在3.2节中我们会详细展开四种交互模式，但这里先给出三条基础原则：

1. **介入点要少而精**。不要在每个AI步骤后都加一个"确认"按钮——那样研究者会陷入确认疲劳。只在AI独立完成度低且出错代价高的步骤设置人工节点。
2. **批量操作优先**。如果研究者需要对50条结果做同样操作（如"全部接受"），应提供批量按钮，而非逐条操作。
3. **修改应能被AI利用**。人的修改不应该只是静态记录，而应该触发AI的重新计算或学习。例如，研究者修正了

一个聚类标签后，AI可以据此调整聚类中心。

实现要点（Vibe Coding提示词模式）

描述交互式架构时，关键是明确UI上哪些元素由AI生成、哪些可由用户编辑、编辑后AI如何响应。一个有效的提示词模板：

> 请用Gradio构建一个界面，实现交互式亲和图功能：
>
> - 左栏显示AI建议的分组结果（每个分组显示名称及其下的条目列表）
> - 右栏为编辑区，用户可以：
>   - 将条目从一个分组拖拽到另一个分组
>   - 双击分组名称进行编辑
>   - 选中两个分组后点击“合并”按钮
>   - 删除某个分组（被删除分组的条目移至“未分组”区）
> - 用户的每次修改自动触发AI重新评估：更新受影响分组的名称建议，并在界面上高亮显示建议变化的区域
> - 所有操作支持撤销/重做

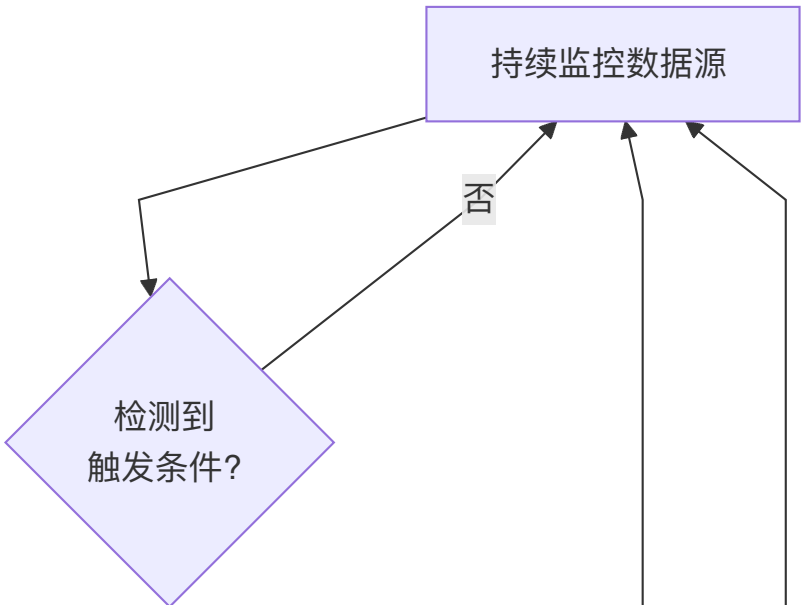
3.1.4 自主代理式架构

定义与结构

自主代理式架构是三种架构中最复杂的一种。它不再是被动响应用户输入的“工具”，而是一个能够主动管理任务流程的**智能代理**。代理式Skill通常具备以下特征：① 持续运行或定期触发；② 能够自主决定何时启动分析、何时请求人类介入、何时发出告警；③ 能够感知环境变化（如新数据到达）并作出响应。

自主代理式架构的结构不是固定的线性流程，而是一个**事件驱动的循环**：

```
监控 → 检测到触发条件 → 执行处理 → 判断是否需要人类介入 →
  如果需要 → 推送通知 → 人类介入 → 继续循环
  如果不需要 → 记录结果 → 继续监控
```



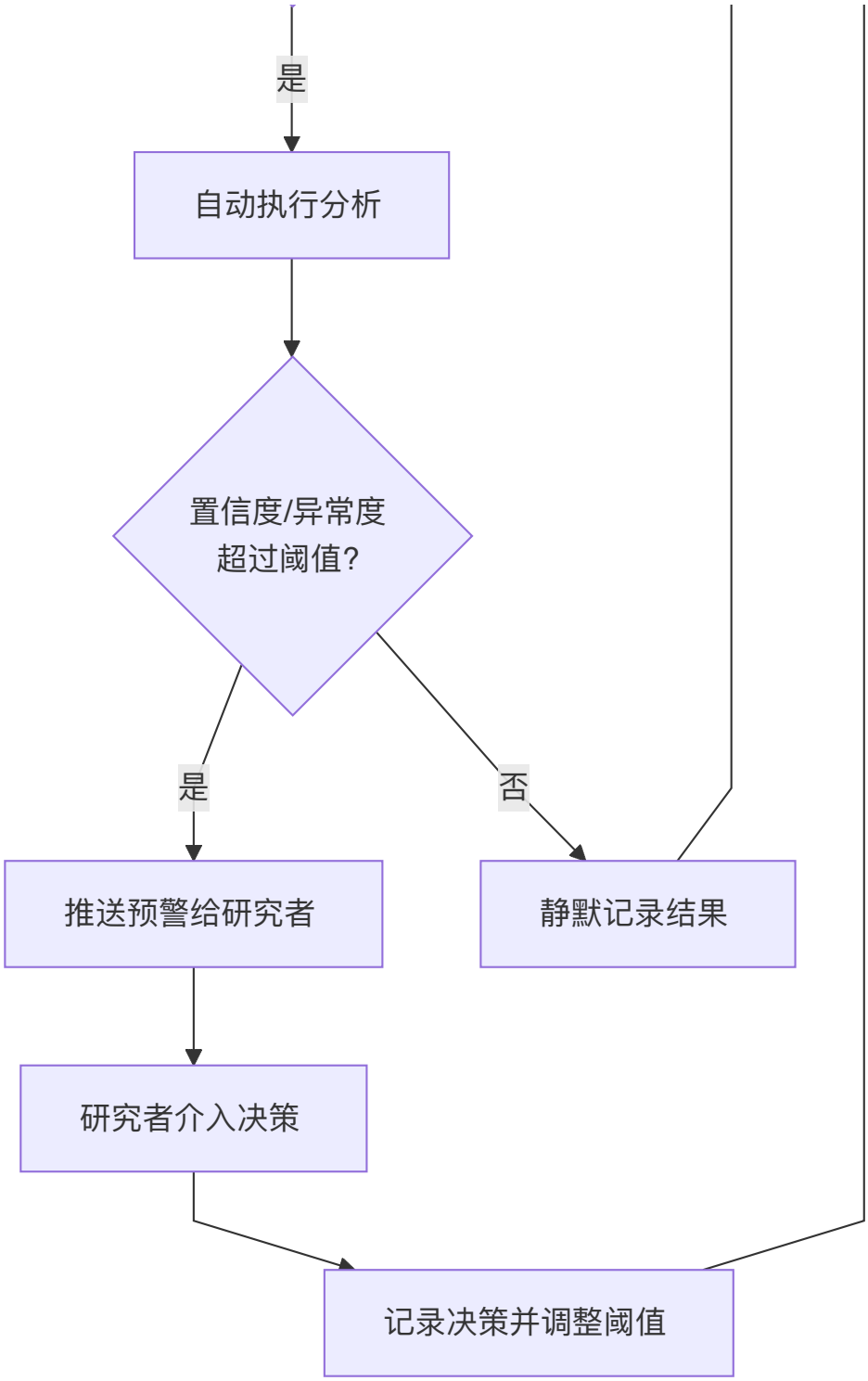


图3-3 趋势监控代理式Skill架构示意图

适用情境

自主代理式架构最适合那些“长期持续、人无法时刻关注、但又不希望错过重要信号”的设计研究任务。最典型的例子是**趋势扫描与情报监控**：AI持续订阅行业新闻、论文预印本、社交媒体讨论，当检测到某话题讨论量突然上升、情感极性反转或出现新兴关键词时，主动向研究者推送预警。

另外一个应用场景是**长期用户体验监控**：Skill持续分析用户反馈流（应用商店评论、客服工单），当某一抱怨主题的频次超过基线两个标准差时，生成问题报告推送给产品团队。

优缺点分析

优点	缺点
解放人的注意力，无需手动启动	实现复杂度显著提高
能够捕捉时间维度上的变化和趋势	调试困难，状态不易追踪
对“未知的未知”有探测能力	误报与漏报的平衡难以把握
提供持续的情报优势	需要处理长期运行中的异常（API变更、网络中断等）

在课程项目中的定位

在8周的课程周期内，完整实现一个自主代理式Skill的难度较高（尤其是需要长期运行的监控部分）。因此，我们的建议是：**优先实现一个交互式或管道式的核心功能，并在设计文档中标注“未来可扩展为代理式”的路径**。例如，一个“手动触发的趋势分析工具”可以平滑扩展为“定期自动运行的预警系统”；一个“单次访谈分析工具”可以扩展为“持续监听新访谈文件并自动处理的代理”。

这种“先做核心，后谈代理”的策略，既保证了项目的可交付性，又展示了架构设计的远见。

实现要点（简化版Vibe Coding提示词）

如果你希望尝试代理式架构的某些元素（如定时触发），可以使用以下提示词：

> 请写一个Python脚本，实现简单的定时任务：每天早上8点，从指定的RSS源抓取最新的10条设计类文章，调用LLM生成每篇文章的摘要，并与历史摘要比较，如果发现新主题在最近7天内出现≥3次，则通过邮件API发送预警邮件到指定地址。脚本需要包含完整的错误处理和日志记录。

3.1.5 架构选择的决策框架

为了帮助小组在正式开发前做出清晰的架构决策，我们提供一个四步决策框架。请结合第二章的拆解文档，逐项评估。

第一步：列出方法中的所有步骤（从第二章的拆解文档中复制）

第二步：对每个步骤进行双维度评分

评分维度	评分标准
AI独立完成度 (1-5)	1=完全不能，需要专家判断；3=半自动，AI提供建议但需人确认；5=完全可以，AI表现稳定
出错代价 (1-5)	1=几乎无影响；3=可能影响下游但可被后续纠正；5=严重影响最终输出且难以发现

第三步：根据评分综合选择架构

- 如果所有步骤的AI完成度≥4 **且** 出错代价≤2 → 可以考虑**管道式**
- 如果存在步骤的AI完成度≤3 **且** 出错代价≥3 → 必须采用**交互式**（在这些步骤设置人工节点）
- 如果任务需要持续监控、主动触发、且处理大量动态数据 → 考虑**代理式**（但建议先代理后扩展）

第四步：对于交互式架构，进一步确定人工节点的具体位置和交互模式

- 管道式部分的最后一个环节结束时（最天然的整体检查点）

- 完成度最低且出错代价最高的步骤之后（风险最高的节点）
- 最终输出交付之前（无论如何都要有的终审节点）

3.2 人类在环：交互模式与设计原则

3.2.1 为什么需要“人类在环”?

在3.1.3节中，我们强调了交互式架构中人类介入的价值。但“人类在环”不仅仅是一个设计选择——它有着深刻的哲学与方法论基础。设计研究的一个核心传统是**诠释学**：研究者不是被动地“反映”数据中的客观事实，而是在特定的理论框架、项目背景和个人经验下，主动地“建构”意义。数据不会自己说话，是研究者通过编码、聚类、命名、叙事，将原始材料转化为可行动的洞察。

AI可以加速这个过程，但它无法替代研究者对意义的最终负责。原因有三：

1. **隐性知识**：波兰尼指出，我们知道的远比我们能说出来的多。研究者对用户微妙的情绪、语境中的弦外之音、设计机会的直觉判断，很难被完全编码为规则。
2. **价值判断**：什么是“重要”的发现？什么是“应该优先解决”的痛点？这些问题涉及价值选择，而AI没有价值立场。
3. **专业责任**：设计研究结论会影响产品决策，产品决策影响真实用户。研究者需要为最终的输出署名，因此必须保留审查和修改的权利。

因此，“人类在环”不是权宜之计，而是设计研究AI Skill的本质要求。我们的目标不是消除人类，而是让人类做更高级、更有创造性的工作。

3.2.2 四种人类在环的交互模式

人类在环并非一个简单的“有/无”开关，而是一个连续谱。根据人类介入的时机、频率和深度，我们可以识别出四种典型的交互模式。

模式一：审核式（Review）

定义：人类作为“审核者”。AI先完成全部处理，将结果提交给人类，人类逐条或批量审核，通过或驳回。驳回时可以修改AI的结果，也可以要求AI重新处理。

适用场景：AI产出整体质量较高，但偶有错误需要人工纠正。例如AI生成的编码建议、主题分组、报告草稿。

交互设计要点：

- 提供高效的批量审核界面（例如，一个“全部接受”按钮，以及高亮标注“AI置信度低”的条目）
- 驳回操作应尽可能低成本（一键拒绝比编辑修改更快，编辑修改比重新指派更快）
- 人的审核轨迹应被记录和可回溯，以便后续分析人类-AI分歧

Vibe Coding描述示例：

> 界面显示AI生成的情感分析结果列表。每条结果旁有一个复选框和一个下拉菜单（正确/错误）。顶部有一个“通过所有高置信度条目”按钮。所有被标记为错误的条目会进入下方的编辑区，用户可以修改情感标签。提交后，系统记录修改历史并输出修正后的数据集。

模式二：引导式（Guide）

定义：人类作为“引导者”。人在AI开始处理之前就设定了方向、约束或优先关注领域。AI在这个框架内进行分析。

适用场景：人对分析目标有清晰的方向，需要AI帮助实现。例如，“重点关注包含‘价格’和‘竞品’的反馈段落”“优先聚类与安全相关的用户行为”。

交互设计要点：

- 提供明确的“引导输入”入口（如关键词输入框、优先级滑块、关注点选择器）
- 将人的引导指令以可见的方式嵌入处理结果（让用户看到“因为您关注了X，所以此处被重点分析”）
- 允许人在处理中途调整引导参数，并实时看到分析结果的变化

Vibe Coding描述示例：

> 界面顶部设置一个文本输入框，用户可以输入一组关注关键词（如“加载速度、闪退、兼容性”）。点击“分析”后，AI在处理用户反馈时，对所有包含这些关键词的段落进行深度情感分析和根因提取，并在输出中将这些段落高亮标记，单独生成“重点关注问题摘要”。

模式三：协作式（Collaborate）

定义：人类作为“协作者”。人和AI几乎同时工作，人的每一次操作都可能触发AI的即时响应。这是交互式架构中最紧密的协同形式。

适用场景：任务有高度互依性，人的一个决定会显著改变AI的后续处理。例如亲和图分组——人调整一个条目的归属后，AI重新计算受影响区域的分组建议；用户画像修订——人修改了某Persona的年龄属性，AI自动调整其行为模式和痛点的权重。

交互设计要点：

- 响应必须即时（人的操作到AI反馈的延迟应小于1-2秒，否则协作感会被破坏）
- AI的响应应“聚焦”而非“全量”——不要每次修改都重新计算整个数据集，而是只更新受影响的局部
- 人应该能够随时“锁定”已经确认的部分，防止AI的后续更新覆盖人的决定

Vibe Coding描述示例：

> 界面左侧显示AI生成的用户画像卡片。用户可以修改卡片的任何字段（年龄、职业、目标、痛点）。每次修改后，右侧的“行为模式”和“需求优先级”模块自动更新——调用大模型API根据修改后的字段重新生成相关内容，并在生成过程中显示加载动画。用户修改超过3次后，系统提示“是否固化当前版本？”以防止无限循环。

模式四：申诉式（Appeal）

定义：人类作为“申诉受理者”。AI自主运行，只有在遇到“低置信度判断”或“需要人的判断力”时才主动打断流程，请求人类介入。

适用场景：AI在大多数时候能够独立运行，但偶尔遇到不确定的情况。例如大规模自动编码中，AI将置信度>0.9的段落自动编码，将置信度在0.5-0.9之间的段落提交给人判断，将置信度<0.5的段落标记为“待人工审核”。

交互设计要点：

- 申诉的内容必须清晰：AI为什么不确定？有哪些候选方案？AI倾向于哪一个？
- 申诉的频次必须可控：如果每处理10条就申诉一次，人就变成了AI的监工，而不是协作者
- 人在申诉点上的决定应该被AI学习，用于减少未来类似情况的申诉次数

Vibe Coding描述示例：

> 系统在后台自动处理用户反馈的情感分类。当遇到置信度低于0.7的案例时，该案例被推送到界面中央的“待确认”面板，并显示AI的困惑说明：“这条反馈可能属于‘中性’或‘负面’，因为包含‘还行’这样的模糊词。请人工判断。”用户做出选择后，系统将该案例及其正确答案加入缓存，后续遇到语义相似的反馈时直接使用该判断。

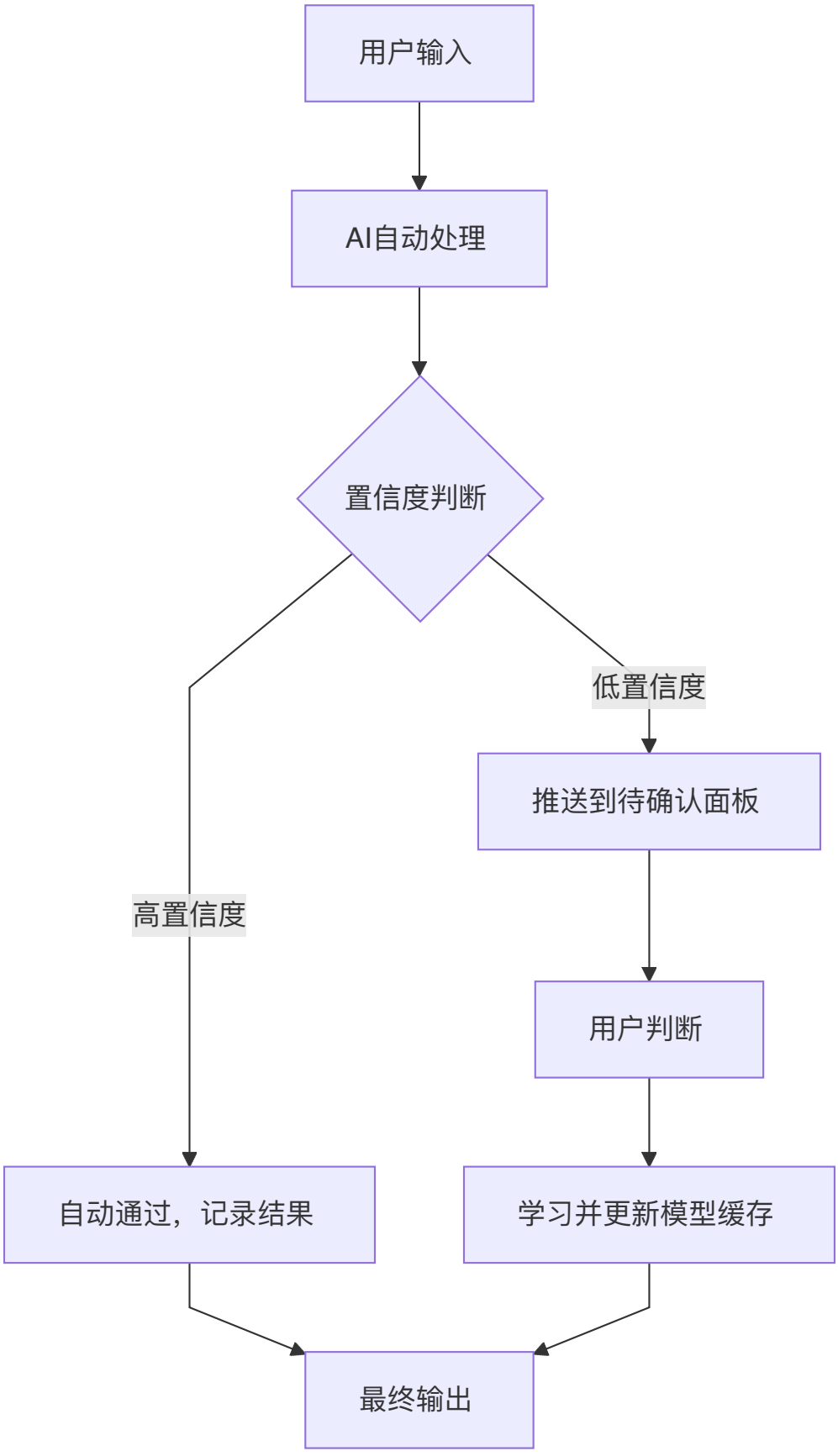


图3-4 申诉式交互模式流程图

3.2.3 交互模式的选择指南

在实际的Skill设计中，你可以混合使用上述四种模式。例如，一个访谈分析Skill可能采用：预处理阶段用**管道式**自动完成；情感分析阶段用**审核式**让人批量确认；主题提取阶段用**协作式**让人拖拽调整；关键段落采用**申诉式**仅在低置信度时询问人。

一个简单的决策表可以帮助你快速选择：

条件	推荐模式
AI输出质量高，错误容易被发现	审核式（批量通过 + 高亮异常）
用户有明确的分析焦点	引导式（关键词/筛选条件）
任务具有空间/结构依赖性（如分组、排序）	协作式（拖拽 + 即时响应）
AI在多数情况下可靠，但偶尔不确定	申诉式（仅在低置信度时打断）

3.3 AI编排基础：链式提示、工具调用与状态管理

在第2周的第3课中，你已经通过Vibe Coding生成了一个简单的多步骤脚本。但那个脚本通常只是“顺序调用几个函数”，缺乏更高级的编排技术。本节将系统介绍三种AI编排的核心技术——**链式提示**、**工具调用**与**检索增强生成（RAG）**，以及**状态管理**。这些技术将帮助你设计更复杂、更可控的处理链。

3.3.1 链式提示

定义与原理

链式提示是编排的最基本形式：将大模型的输出设计为下一个提示词的一部分。每一次API调用不只是一个独立的问答，而是一个链条中的一环。

为什么需要链式提示？原因有三：

1. **上下文窗口限制**：一个提示词中塞入太多指令和信息，可能超出模型的上下文窗口（尽管现代模型窗口已非常大，但过长的提示词仍可能导致注意力稀释）。
2. **任务分解提高准确率**：将复杂任务拆解为多个简单子任务，每个子任务让模型专注于一件事，整体准确率往往高于“一步到位”。
3. **可调试性**：链式提示将中间输出暴露出来，你可以检查每一步的结果，定位错误发生在哪个环节。

三种链式模式

A. 顺序链：最直接的链式——步骤A的输出直接作为步骤B的输入的一部分。

示例（访谈主题提取）：

- 步骤1提示：“将以下访谈文字稿分成段落，标注每段的核心话题（5字以内）。输出JSON列表。”
- 步骤2提示：“以下是标注了核心话题的段落列表：[步骤1的输出]。请将这些话题聚类为3-5个主题，每个主题给出名称和涵盖的段落编号。”

B. 分支链：根据条件选择不同的后续路径。

示例（用户反馈处理）：

- 首先判断情感极性。
- 若正面："提取用户最满意的2个功能点。"
- 若负面："分析用户的挫折来源，并给出改进建议。"
- 若中性："询问用户是否有更多细节补充，并标记为待跟进。"

C. 汇总链：多个并行的链处理完后，由一个汇总链综合。

示例（多份访谈综合分析）：

- 并行处理：对5场访谈分别运行"提取关键主题和代表性引语"。
- 汇总提示："以下是5场访谈提取的主题摘要：[各链的输出]。请综合为一个统一的主题结构，标注每个主题的来源场次和场间一致程度。"

链式提示的Vibe Coding描述模板

在Vibe Coding中，你不需要手动实现链的逻辑——AI会为你生成代码。但你需要清晰地向AI描述"这是一个多步骤的链"。以下是有效模板：

> 我需要一个多步骤的分析流程，请将任务拆分为独立的函数，并用状态传递中间结果。
>
> 步骤1：输入原始文本列表，调用DeepSeek API判断每条文本的情感（正面/负面/中性），输出字典`{text: sentiment}`。
> 步骤2：读取步骤1的输出，筛选出情感为"负面"的文本，调用API分析每条负面文本的原因（输出"根因"字段）。
> 步骤3：汇总前两步的结果，生成统计报告：负面比例、常见根因词云数据。
>
> 请确保步骤2只处理负面文本，步骤3能够访问步骤1和步骤2的所有输出。

3.3.2 工具调用

定义：工具调用（Function Calling）是让大模型在生成回应的过程中，主动调用外部工具或函数的能力。它使模型不再只是一个"文本生成器"，而是一个"能够触发操作的智能代理"。

工作原理（通俗版）：你在提示词中告诉模型"你可以使用以下工具"。当模型认为当前任务需要某个工具时，它不输出普通文本，而是输出一个结构化的"工具调用请求"。你的程序拦截这个请求，执行对应的函数，将结果传回给模型，模型再将工具返回的结果整合进最终的回复。

设计研究中的应用场景：

- **联网搜索**：在趋势扫描Skill中，让模型搜索最新的行业新闻，而不是只依赖训练数据中的旧知识。
- **数据查询**：在问卷分析中，模型调用统计函数计算p值或效应量，避免自行计算错误。
- **文件操作**：模型调用保存函数，将分析结果写入指定格式的文件。
- **图像分析**：模型调用图像识别工具，分析用户上传的截图中的UI元素。

Vibe Coding中的工具调用描述

在Vibe Coding中，你可以先定义工具函数的大致功能，然后让AI将工具调用整合到处理链中。示例：

> 请写一个访谈分析脚本。除了调用大模型API进行分析外，还需要定义以下工具函数：
>
> - `save_report(content, filename)`：将报告内容保存为Markdown文件
> - `search_web(query)`：使用DuckDuckGo搜索指定关键词，返回前3条结果的摘要
>
> 在分析过程中，如果遇到专业术语（如"认知负荷""Fitts定律"），请调用`search_web`查术语解释并补充到报告中。分析完成后，调用`save_report`保存最终报告。

安全提示：工具调用，尤其是联网搜索和文件写入，可能带来安全隐患（如覆盖系统文件、引入未经验证的信息）。在Vibe Coding中，应要求AI加入安全检查：

> 所有工具调用前需检查：文件写入路径必须在当前工作目录下，不允许出现`..`或绝对路径；联网搜索结果必须标注来源URL和检索时间；如果工具调用失败，程序应优雅降级而非崩溃。

3.3.3 检索增强生成（RAG）与上下文注入

定义：检索增强生成——在让模型回答之前，先从外部知识库中检索与当前问题相关的文档片段，将这些片段作为上下文注入到提示词中，再进行生成。RAG的核心价值是：让模型能够访问训练数据之外的、最新的、特定的领域知识。

设计研究中的典型场景：

- **跨访谈引语搜索**：研究者输入查询（如"用户抱怨数据导出"），系统从所有访谈文字稿中检索最相关的段落，将这些段落连同查询一起发给模型，要求模型基于这些段落生成回答并给出引用。
- **竞品分析**：用户上传多个竞品文档，模型在处理用户问题时能够检索这些文档中的相关内容。
- **启发式评估**：将尼尔森十大原则的详细说明作为背景资料注入，让模型更准确地判断界面是否违反原则。

简化版RAG：上下文注入

完整的RAG实现涉及向量数据库、索引构建、检索算法等，对于编程基础薄弱的学生来说可能过于复杂。但RAG的核心价值可以通过一个简化的方式实现：**上下文注入**。

如果你的文档总量不大（例如5-10场访谈的文字稿），可以在每个分析提示词中，将所有文档作为附件提供（前提是不超出上下文窗口）。如果文档总量较大，可以先用简单的关键词搜索或语义相似度计算缩小范围，然后将检索到的前K个段落注入提示词。

Vibe Coding中的RAG描述

> 我需要一个访谈分析工具，支持"引语搜索"功能。用户输入查询后，系统执行以下步骤：
>
> 1. 使用`sentence-transformers`将所有访谈文字稿按段落切分，并为每个段落生成嵌入向量。
> 2. 将用户的查询也转换为嵌入向量，计算与所有段落向量的余弦相似度，选出相似度最高的前5个段落。
> 3. 将这5个段落（连同它们所属的文件名和行号）作为上下文，与用户的查询一起发送给大模型。
> 4. 大模型必须基于提供的上下文回答，并在回答中标注引用的段落来源。如果上下文中没有相关信息，回答"未找到相关引语"。
>
> 请实现上述流程，使用HuggingFace的`sentence-transformers`进行嵌入计算。

3.3.4 状态管理与上下文传递

定义：状态管理回答的问题是——在不同处理步骤之间，哪些信息需要被记住和传递？一个Skill通常包含多个步骤（无论是AI步骤还是人工节点），后续步骤需要知道前面步骤产生了什么结果、用户做了哪些修改。

设计研究中需要传递的典型状态：

- **数据状态**：原始数据、处理后的数据（如文本段落、情感标签、编码）
- **决策状态**：用户在交互式Skill中的修改记录（如"用户将段落A从分组X移到了分组Y"）
- **会话状态**：对于代理式Skill，需要记住上次运行的时间、哪些告警已发送、新数据是否已处理

Vibe Coding中的状态管理描述

> 请设计一个状态管理模块，使用Python字典作为状态容器。状态应包含以下字段：
>
> - `raw_data`: 原始输入文本列表
> - `sentiments`: 每条文本的情感分析结果（格式：`[{text, sentiment, confidence}]`）
> - `clusters`: 聚类结果（格式：`{cluster_id: [text_indices]}`）
> - `user_modifications`: 用户对聚类结果的修改记录（格式：`{timestamp, action, details}`）
> - `final_report`: 最终生成的报告文本
>
> 每个处理步骤的函数接收状态字典作为参数，返回更新后的状态字典。主函数按顺序调用各步骤，并确保状态字段被正确填充。注释中注明每个函数读取了哪些字段、写入了哪些字段。

3.4 评估框架：从信效度到五维度量

3.4.1 为什么需要单独的评估框架?

在传统的软件开发中，"测试"通常指功能测试：按钮是否响应、计算是否正确、是否崩溃。但在AI Skill中，一个系统可以在"代码完美运行、界面毫无bug"的同时，产出完全错误或严重偏差的分析结果。例如，一个将所有用户反馈都判定为"中性"的情感分析器——代码没有bug，但它没有价值。这种"正确运行不等于正确分析"的现象，要求我们建立一套超越功能测试的评估框架。

另一方面，设计研究者天生熟悉"信度"与"效度"这两个概念。信度问的是"测量的一致性"——不同研究者用同一方法分析同一数据，能否得到相似结论？效度问的是"测量的准确性"——你测量的东西是否真的是你想要测量的概念？这些概念可以迁移到AI Skill的评估中：我们需要评估Skill的**输出一致性**（类似信度）和**输出准确性**（类似效度），但还需要增加AI特有的维度，例如可解释性（用户能否理解AI为何给出这个结果）。

因此，本节提出一个**五维评估框架**：准确性、一致性、可解释性、效率、体验。每个维度都包含具体的评估问题和可行的度量方法。

3.4.2 五维评估框架详解

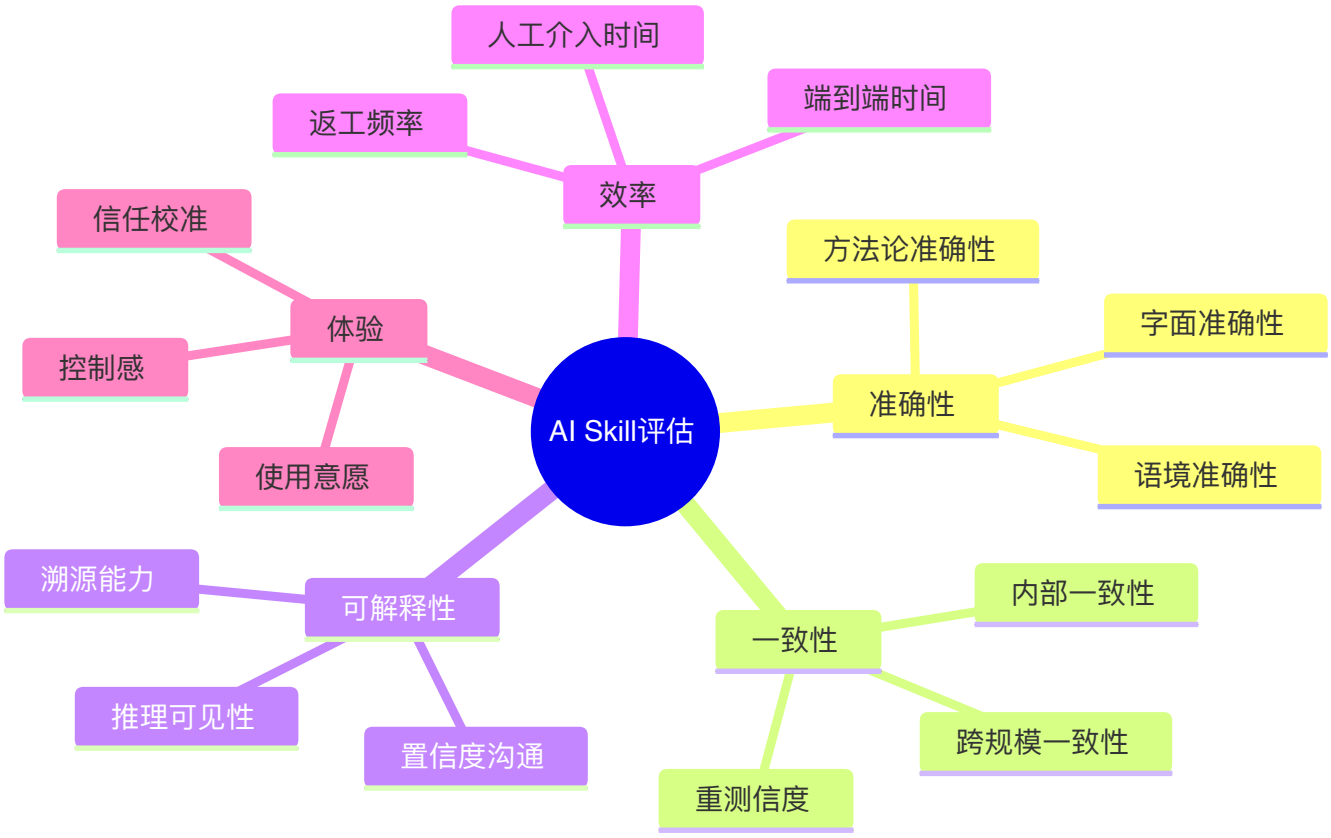


图3-5 五维评估框架思维导图

维度一：准确性

定义：AI Skill的输出与"正确输出"（通常由专家标注提供）之间的匹配程度。

在设计研究Skill中的具体评估问题：

- 情感分析：AI判断的情感极性与专家标注的一致率是多少?
- 主题聚类：AI形成的分组是否在语义上合理？有无明显错分的条目?
- 编码建议：AI生成的初始编码是否覆盖了专家编码中的主要类别?
- 启发式评估：AI检测出的违规点是否真的违反了原则？漏报率多高?

三个层次的准确性：

1. **字面准确性**：正确理解文本的表面含义（如否定句识别）。
2. **语境准确性**：正确理解文本在特定语境中的含义（如"还行吧"在访谈中可能表示不满）。
3. **方法论准确性**：AI的输出是否符合该方法论的标准（如亲和图分组命名是否具有合适的抽象层次）。

度量方法：

- **人机比对**：准备小规模专家标注数据集（20-50条），计算准确率、召回率、F1分数。
- **错误分类分析**：不仅要看错误率，还要分析错误类型（系统性偏差 vs 随机噪声）。
- **边际准确性**：在AI置信度低的子集上单独计算准确性，验证置信度是否可作为自动筛选的条件。

维度二：一致性

定义：AI Skill在相同或相似条件下输出的稳定性——相当于设计研究中的“信度”。

评估问题：

- **重测信度**：同样的数据输入两次，实质性结论是否一致？（注意API可能有随机性，允许措辞差异）
- **内部一致性**：对相似的输入，AI是否给出相似的输出？（例如两个表达近似情感的文本是否得到相同的情感标签）
- **跨规模一致性**：在小样本和大样本上运行，核心洞察的方向是否一致？

度量方法：

- **重测法**：同一数据集运行两次，计算聚类结果的一致性（Adjusted Rand Index）或情感分布的相关性。
- **分半法**：将数据集随机分成两半，比较两半的主题结构是否相似。

AI特有的不一致性来源：

- 大模型API的随机采样（可通过设置`temperature=0`或固定随机种子缓解）
- 第三方模型的无公告更新（需在评估报告中标注模型版本和日期）
- 提示词敏感性（微小措辞变化可能导致输出不同）

维度三：可解释性

定义：用户能否理解AI为什么给出了某个输出——这是设计研究中可追溯性的核心要求。

评估问题：

- **溯源能力**：给定一个结论，用户能否在原始数据中找到支撑证据？
- **推理可见性**：AI是否输出了判断依据？（如“我将这两条分为一组，因为它们共享关键词X、Y，语义相似度0.87”）
- **置信度沟通**：AI是否表达了对自己判断的把握程度？
- **修改追踪**：在交互式Skill中，用户能否区分哪些部分是AI生成的、哪些是用户修改过的？

度量方法：

- **逆向追溯测试**：给定AI的一个结论（如“主题3：数据导出困难”），让用户在原始数据中定位支持该结论的引语，记录平均定位时间。
- **思考出声法**：请测试用户在使用Skill时出声思考，记录他们在哪些节点感到困惑。
- **可解释性自评量表**：后测问卷中包含“我理解为什么系统做出了这个分析”（1-5分）。

设计策略（不依赖复杂技术）：

- 每条AI结论旁附带**引用链接**，点击跳转到原始数据。
- 用**颜色深浅**或**图标**表示置信度（✅高置信度，⚠️中置信度，❓低置信度）。
- 提供“**为什么**”展开区，展示AI的推理过程。
- 在交互式Skill中保留**修改历史侧栏**。

维度四：效率

定义：使用AI Skill相比传统方法节省的时间和精力。

评估问题：

- **端到端时间**：用户从提供输入到获得可用输出需要多长时间？与纯人工相比节省了多少？
- **人工介入时间**：在交互式Skill中，用户在人工节点上花费的总时间是多少？
- **返工频率**：用户需要重复操作或回到上一步的比率有多高？

度量方法：

- **对比计时实验**：同一任务分别用Skill和纯人工方法完成，记录总耗时和主观疲劳度（每组5-10名被试）。
- **操作日志分析**：记录每个环节的停留时间、修改次数、撤销次数等。
- **任务完成率**：给定标准任务，限定时间内完成的比率。

"自动化悖论"的警示：
效率评估中最容易被误判的是：AI自动处理很快，但人工审查的时间可能比纯人工处理更长（因为审查AI的输出需要逐条比对）。因此，**效率度量必须是端到端的**，而不是只看AI处理环节。

维度五：体验

定义：用户在使用Skill过程中的主观感受——满意度、信任、控制感、心理负荷。

评估问题：

- **信任校准**：用户对AI输出的信任度是否与AI的实际准确率匹配？（过度信任或信任不足都是问题）
- **控制感**：用户是否感觉自己掌控着分析的方向，而非被AI推着走？
- **使用意愿**：用户下次有类似任务时，是否愿意再次使用该Skill？
- **学习成本**：用户从初次接触到熟练使用需要多长时间？

度量方法：

- **标准化量表**：System Usability Scale（SUS）、NASA-TLX（任务负荷）、Trust in Automation Scale。
- **半结构化访谈**："使用过程中最让你困惑/满意/挫折的是什么？"
- **行为观察**：用户是否需要额外的指导或反复尝试？

体验维度的核心：信任校准
设计研究者对自己的分析产出负有专业责任。如果Skill让研究者过度信任AI的输出（"AI说这是核心痛点，那应该没问题"），而输出实为幻觉，后果比效率低下更严重。因此，在体验评估中，不仅要问"用户满不满意"，还要设计机制确保用户保持适当的警惕性——例如，在低置信度输出上强制展示"建议人工复核"提示。

3.4.3 将评估嵌入Skill开发流程

评估不应是项目结束时的"一次性测试"，而应贯穿整个开发周期。建议采用以下节奏：

阶段	评估重点	方法
第4周 MVP	技术可行性、关键路径准确率	快速人机比对（10条数据）
第5周 迭代	准确性、一致性	扩展数据集（30-50条），重复测试
第6周 测试	五维全面评估	用户测试（3-5人），量表+日志+访谈
第7周 优化	针对性改进	对低分维度进行专项优化
第8周 交付	最终评估报告	汇总所有数据，形成结论

实现评估功能的Vibe Coding描述：

> 请为我的Skill添加一个评估模式，当设置`eval_mode=True`时：
>
> - 对输入数据中的每条，同时记录AI输出和用户（专家）标注的答案
> - 计算并输出准确率、召回率、F1分数，按类别分别展示
> - 生成一个差异报告，列出所有AI与专家不一致的案例，附上原始文本、AI输出和专家答案
> - 将评估结果保存为JSON文件，并生成一个简单的HTML可视化页面

3.5 设计文档与原型：从蓝图到验证

3.5.1 Skill设计文档结构

经过前四节的讨论，你已经具备了撰写一份完整Skill设计文档所需的所有知识。本节给出一个标准文档模板，建议按此结构组织，但允许根据具体方法进行调整。

第一章：方法与范围

- 所选方法简述（1段）
- 重构范围：本次重构覆盖该方法的哪些环节？哪些环节不覆盖？为什么？
- Skill一句话定义（例如：“一个辅助设计研究者进行亲和图分析的AI工具，自动建议分组和命名，支持拖拽调整”）

第二章：Skill架构

- 架构模式（管道式/交互式/代理式）及选择理由
- 处理流程架构图（可使用Mermaid或UML图），标注每个节点的AI/人工角色、数据流向
- 人工节点详情：位置、交互模式（审核/引导/协作/申诉）、操作方式

第三章：技术架构

- 处理链描述：输入 → 步骤$_1$ → 步骤$_2$ → … → 输出，每步标注使用的AI能力（如“文本嵌入+DBSCAN”）
- 技术栈选择：LLM API（哪个模型）、Python库（sentence-transformers、Gradio等）、框架（LangChain或纯Python）
- API候选：至少2个适合独立封装为API的子功能，标注端点名、输入、输出、调用场景

第四章：交互设计

- 目标用户画像：谁会用这个Skill？他们有什么特征？
- 使用场景：用户在什么情境下打开这个Skill？期望得到什么？
- 关键交互流程（附界面草图或Gradio原型截图）
- 界面布局说明（可用文字描述或标注图）

第五章：数据策略

- 开发阶段数据来源与量级（真实/模拟，多少条）
- 测试阶段数据来源与量级
- 数据隐私与伦理：如果使用真实用户数据，是否已脱敏？是否有数据使用协议？

第六章：评估策略

- 五维评估的具体方案（直接引用3.4节设计的评估方案）
- 测试时间线与资源需求（需要多少被试？多少标注工作量？）

第七章：风险与缓解

- 关键风险（技术、数据、时间、团队）及缓解策略
- 如果第4周MVP遇到重大障碍，备选方案是什么？

文档篇幅建议：正文2000-4000字（不含图和附录）。

3.5.2 Gradio原型：交互验证的最轻量方式

为什么用Gradio?
Gradio是一个用Python快速构建机器学习Web界面的库。它可以在几行代码内生成一个带有文件上传、输出显示、交互组件的界面，无需编写HTML/CSS/JavaScript。对于非程序员出身的设计研究生，Gradio是验证AI Skill交互设计的理想工具。

原型的目标不是"完整功能"，而是"验证交互流"。一个有效的原型应该：

- 展示输入方式（文件上传、文本框、下拉选择等）
- 展示处理过程（可加进度条或状态文字）
- 展示输出结果（文本、表格、JSON、图片等）
- 如果可能，展示人工操作节点（如审核按钮、拖拽区域）

即使后端数据是假的（mock），原型依然有价值。你可以在原型中硬编码一些示例输出，让用户体验交互流程，收集反馈。等交互设计稳定后，再接入真正的AI处理链。

用Vibe Coding生成Gradio原型：

> 请用Gradio构建一个交互式原型，实现亲和图辅助工具的界面：
>
> - 输入：一个文件上传组件，支持上传TXT或CSV，每行一条短文本（便利贴内容）
> - 处理：点击"分析"按钮后，调用一个mock函数（先不接入真实AI），模拟生成3个分组，每个分组有名称和包含的条目列表

> - 输出：左侧显示分组结果，右侧显示该分组的详细内容。用户可以：
>   - 编辑分组名称（文本框）
>   - 将条目拖拽到其他分组（使用Gradio的Dataframe或List组件）
> - 底部有一个“导出为JSON”按钮，将当前分组结构保存为文件
>
> 请在代码注释中标注“TODO: 替换mock函数为真实AI聚类”。

3.5.3 从原型到MVP的演进路径

原型验证通过后，你可以按照以下路径逐步替换mock为真实处理逻辑：

1. **第一步**：接入一个最简单的AI处理（例如，对单条文本做情感分析），验证API调用和界面反馈。
2. **第二步**：实现核心的AI处理（如嵌入+聚类），并让界面真实调用它。
3. **第三步**：加入人工节点交互（如拖拽调整、编辑名称），实现状态的保存和传递。
4. **第四步**：加入评估日志记录，开始收集准确性数据。
5. **第五步**：优化性能和精度，准备用户测试。

在整个演进过程中，保持与AI的持续对话：每当需要修改或增加功能，就用自然语言向AI描述，让它生成代码修改。

3.6 本章小结

本章是《智能设计系统》教材中承上启下的关键一章。我们从架构设计出发，建立了三种核心架构模式（管道式、交互式、自主代理式）及其选择框架，并深入探讨了“人类在环”的四种交互模式（审核、引导、协作、申诉）。这些设计模式将抽象的方法拆解转化为可落地的系统蓝图。

在编排技术部分，我们介绍了链式提示、工具调用、RAG和状态管理，掌握了用自然语言描述复杂AI处理链的方法。这些技术不仅是实现Skill的工具，更是你与AI协作开发时的“沟通语言”——当你能够精确地说出“步骤A的输出是步骤B的输入的一部分，步骤C仅在条件D下执行”，AI就能生成期望的代码结构。

在评估框架部分，我们建立了五维评估体系，将设计研究中的信效度概念迁移到AI技能的测试中。这一框架不仅用于最终的评价，更应在开发早期就嵌入Skill的设计——一个从一开始就考虑可解释性的Skill，远比后期补救要优雅得多。

最后，通过设计文档模板和Gradio原型指导，我们完成了从“蓝图”到“可验证模型”的闭合。第4周开始，你将正式进入MVP开发冲刺——而你已经握有清晰的架构、编排方案和评估标准，可以自信地驶入开发期。

思考与练习

一、概念理解题

1. 请用你自己的语言解释管道式架构与交互式架构的本质区别。在什么情况下必须选择交互式而非管道式？
2. “人类在环”的四种交互模式（审核、引导、协作、申诉）分别适用于什么样的设计研究场景？请各举一例。
3. 什么是链式提示？为什么相比“一个提示词完成所有事”，链式提示在复杂任务中往往准确率更高？
4. 五维评估框架中，“可解释性”和“体验”维度有何关联与区别？为什么设计研究Skill对可解释性有特殊要求？

二、分析应用题

5. 以下是一个设计研究方法的重构描述，请分析它采用了什么架构模式？指出其中哪些是AI步骤，哪些是人工节点，并评估该设计是否合理。

> “用户上传访谈录音后，系统自动转录为文字（Whisper），然后自动进行情感分析和主题聚类。聚类结果在界面左侧显示为树形结构，用户可以拖拽调整节点位置、修改节点名称。调整后，系统根据用户的修改重新计算主题间的相关性，并更新树结构。最终用户点击‘生成报告’按钮，导出分析结果。”

6. 假设你正在开发一个**访谈分析Skill**，请为以下三个环节分别选择最合适的交互模式，并说明理由：
    - 对每段文本的情感极性进行初步判断
    - 对AI生成的6个主题名称进行审核
    - 处理AI遇到的不确定案例（置信度<0.6）

三、实践与反思题

7. 以小组为单位，使用3.5节的文档模板撰写你们的Skill设计文档v1.0，并搭建一个Gradio原型（可以先用mock数据）。然后在课堂上进行5分钟演示，记录导师和同学的反馈意见，并在文档中补充修改计划。
8. 基于你们的Skill设计，完成五维评估方案的初稿。具体要求：每个维度至少设计1个可操作的度量指标，并说明需要多少测试数据/被试、预计耗时。提交评估方案。
9. **个人反思**（500字）：在完成本章的学习和小组设计后，你认为自己在“将设计研究方法转化为AI Skill”这件事情上，最大的认知突破是什么？最让你感到不确定或困难的地方又是什么？你对第4周开始的MVP开发有哪些具体准备？

参考文献

[1] Amershi, S., et al. (2019). Guidelines for human-AI interaction. *Proceedings of CHI 2019*.

[2] Dell’Acqua, F., et al. (2023). Navigating the jagged technological frontier. *Harvard Business School Working Paper*, No. 24-013.

[3] Karpathy, A. (2024). Vibe coding: A new way to program. *Personal blog / X*.

[4] Lewis, P., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459-9474.

[5] Liu, P., et al. (2023). Pre-train, prompt, and predict: A systematic survey of prompting methods in NLP. *ACM Computing Surveys*, 55(9), 1-35.

[6] Polanyi, M. (1966). *The Tacit Dimension*. University of Chicago Press.

[7] Sauro, J., & Lewis, J. R. (2016). *Quantifying the user experience: Practical statistics for user research*. Morgan Kaufmann.

[8] Weisz, J. D., et al. (2021). Design principles for generative AI applications. *Proceedings of CHI 2021*.